The Definitive Guide to SQLite




# GROUP 4

Chou Yamata
Hak Johnpeter
Pen Punleu
Rithy seavthean
Thoeun Sophal
Try LeangSreng
Chem sombok
Wathana Panhanuth

MADE BY CHOU YAMTATA

**Introduction**

- Key characteristics of SQLite

**What is SQLite?**

- Serverless
- Self-Contained
- Zero-configuration
- Transactional
- SQLite unique features
- When to use SQLite
- Advantages
- Disadvantages

**Installing SQLite**

- Basic SQLite Commands

**SQLite Sample Database**

- Chinook sample database tables
- How to connect to SQLite sample database

**SQLite Commands**

- Opening sqlite3 tool
- Getting helps
- Quitting sqlite3 tool
- Creating a new SQLite database
- Opening a SQLite database file using sqlite3
- Displaying the current database
- Showing tables in a database
- Showing table structures
- Showing indexes
- Executing an SQL statement
- Formatting query output
- Escaping the …> in the middle of the query
- Saving the result of a query into a file
- Executing SQL statements from a file

**Common Use Cases of SQLite (What it is usually used for)**

1. Mobile Applications
2. Desktop Applications
3. Embedded Systems and IoT Devices
4. Standalone Applications and Local Tools
5. Web Development Prototyping

6. Data Analysis and Local Data Storage
7. File-Based Data Formats
8. Applications Needing Offline Capability
9. Small Websites and Backend Systems

**When not to use SQLite**

1. Working with lots of data
2. High write volumes
3. Network access is required

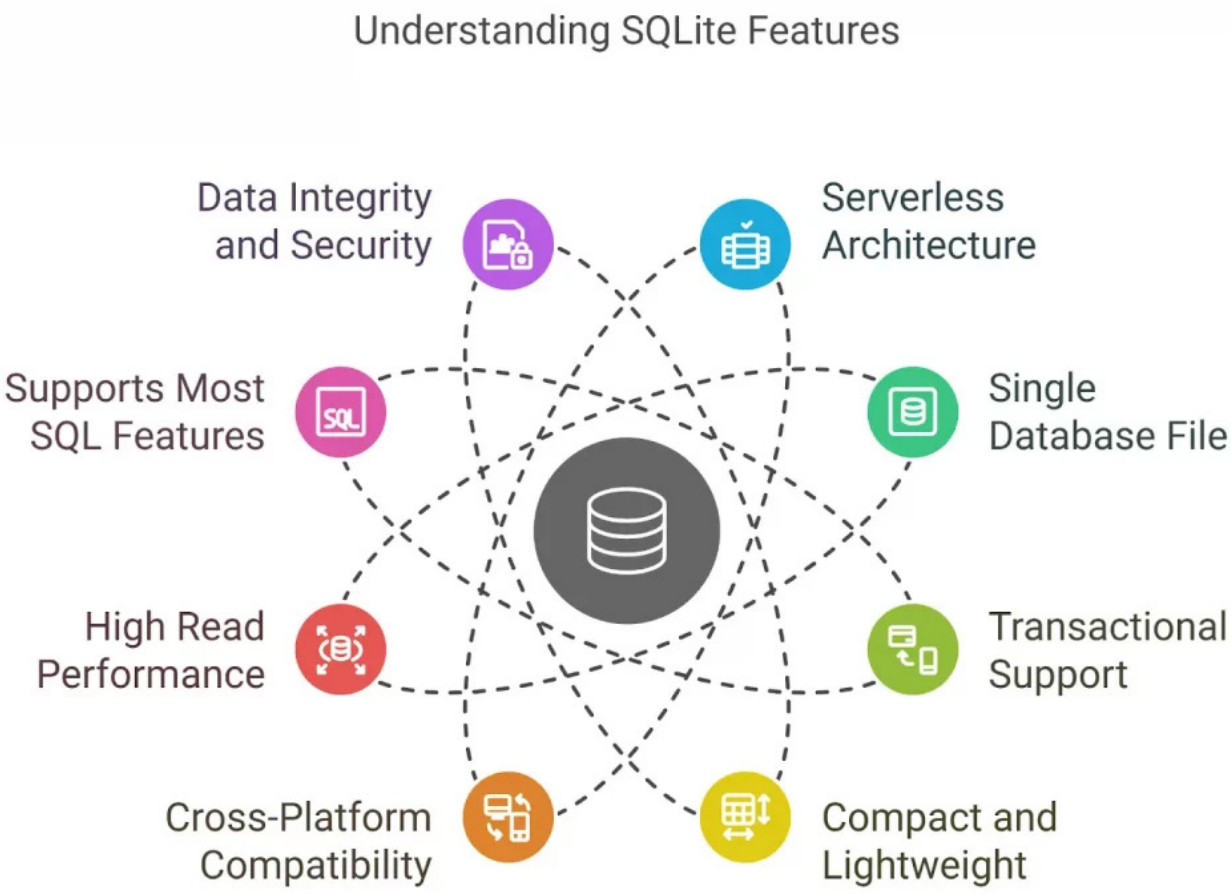
**What is SQLite used for**

**When to use MySQL vs PostgreSQL vs SQLite**

**Conclusion & Summary**

# Introduction

SQLite is a free and open-source relational database management system (RDBMS) implemented as a C-language library. It is distinguished by its serverless architecture, meaning it operates without a separate server process. Instead, it directly reads and writes to ordinary disk files. A complete SQLite database, including multiple tables, indices, triggers, and views, is contained within a single disk file.




**Key characteristics of SQLite:**

Serverless: Unlike most other SQL databases (e.g., MySQL, PostgreSQL), SQLite does not have a separate server process. The database engine is directly integrated into the application that uses it.

Self-contained: A complete SQLite database, including multiple tables, indices, triggers, and views, is contained within a single disk file.

Zero-configuration: There is no separate installation or setup procedure required to initialize SQLite before use, and no configuration files are needed.

Transactional: SQLite supports ACID (Atomicity, Consistency, Isolation, Durability) compliant transactions, ensuring data integrity and reliability, even in the event of system failures.

Lightweight and embedded: Due to its small footprint and minimal resource requirements, SQLite is ideal for embedded systems, mobile devices (e.g., smartphones), desktop applications, and local data storage within larger applications like web browsers.

# What is SQLite?

SQLite is a self-contained, lightweight, and serverless relational database management system that stores an entire database in a single file on disk. Unlike client-server databases, it does not require a separate server process, making it a popular choice for embedded systems, mobile apps, and small-scale applications because of its simplicity, minimal configuration, and portability. It is a C library that developers embed directly into their applications to handle data using SQL.

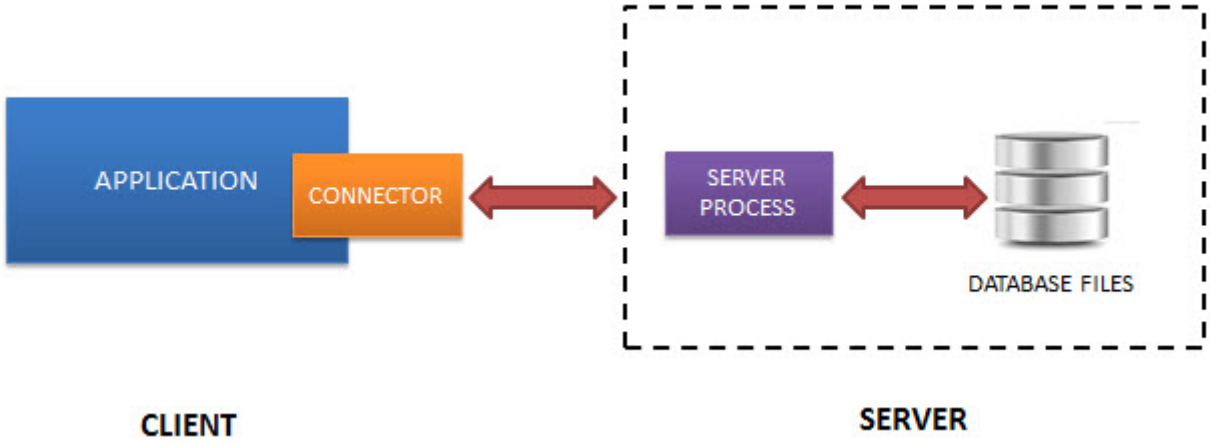
SQLite has the following notable features:

- Self-contained
- Serverless
- Zero-configuration
- Transactional

## Serverless

SQLite is a self-contained, lightweight, and serverless relational database management system that stores an entire database in a single file on disk. Unlike client-server databases, it does not require a separate server process, making it a popular choice for embedded systems, mobile apps, and small-scale applications because of its simplicity, minimal configuration, and portability. It is a C library that developers embed directly into their applications to handle data using SQL.

The following diagram illustrates the client/server architecture of an RDBMS:

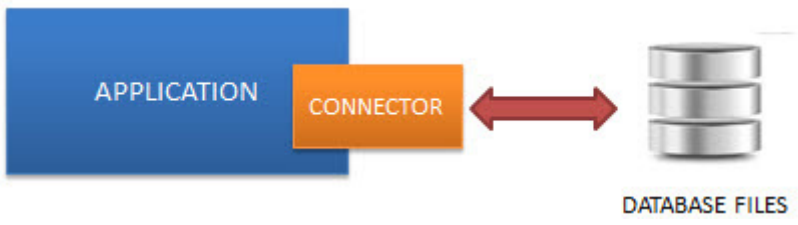



SQLite does NOT work this way.

SQLite does NOT require a server to run.

SQLite database is integrated with the application that accesses it. The application interacts with the SQLite database by reading and writing directly from the database files stored on disk.

The following diagram illustrates the server-less architecture of SQLite:



### Self-Contained

SQLite is self-contained, requiring minimal support from the operating system or external libraries. This makes SQLite suitable in any environment, especially in embedded devices like smartphones, game consoles, handheld media players, etc.

SQLite is developed using ANSI-C and the source code is available as a single file **sqlite3.c,** along with its header file **sqlite3.h.**

If you want to develop an application that uses SQLite, you can drop these files into your project and compile them with your source code.

### Zero-configuration

Thanks to SQLite's serverless architecture, you don't need to "install" SQLite before using it. There is no need to configure, start, or stop a server process. Additionally, SQLite does not require any configuration files.

### Transactional

All transactions in SQLite are fully ACID-compliant, meaning that all queries and changes are Atomic, Consistent, Isolated, and Durable.

In other words, all changes within a transaction take place completely or not at all, even when an unexpected situation like an application crash, power failure, or operating system shut down.

### SQLite unique features

SQLite uses dynamic types for tables, meaning you can store any value in any column, regardless of the declared data type.

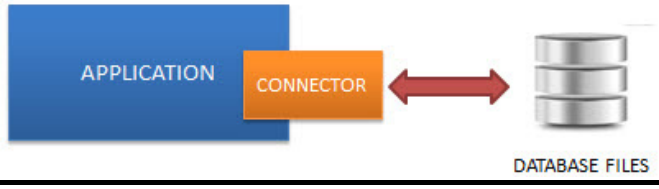
SQLite allows a single database connection to access multiple database files simultaneously. This feature allows you to join tables in different databases or copy data between databases with a single command.

Additionally, SQLite can create in-memory databases, which is very fast for prototyping and testing.

### When to use SQLite

When you need simplicity, speed, and minimal resources, you might consider SQLite. For example:

- **Embedded Apps:** SQLite is ideal for apps that need to store data locally without the overhead of separating database servers such as mobile apps and desktop software.
- **Local storage:** SQLite is suitable for apps that need to store user preferences, settings, or cached data locally.
- **Cross-platform apps:** Since SQLite runs on multiple platforms, it is suitable for apps that need to run cross-platform.
- **Prototyping and development:** SQLite does not require a complex setup, making it ideal for quick prototyping, development, and testing.
- **Internet of Things (IoT) devices:** IoT devices have limited resources. SQLite is very lightweight and is suitable for IoT devices.



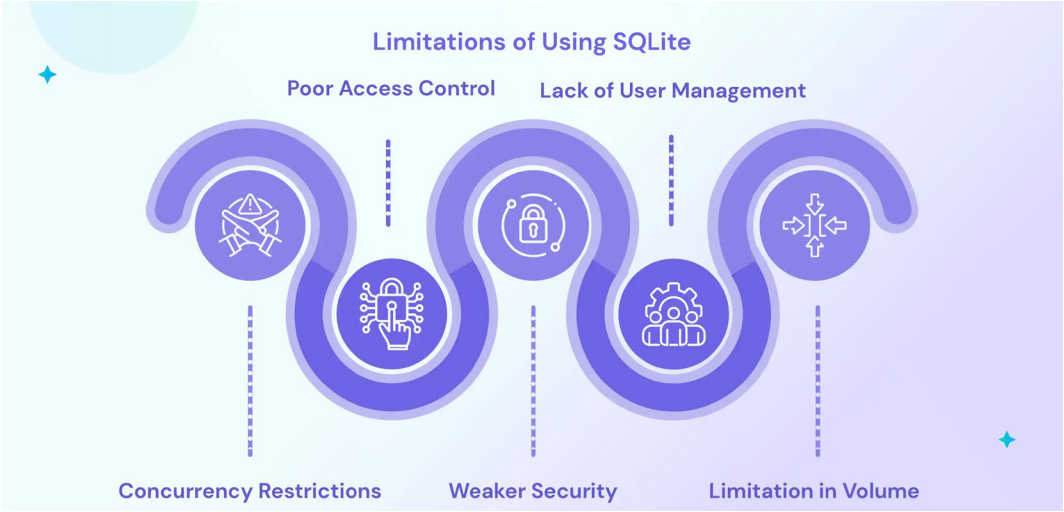



**Advantages**

- Serverless
- Portable — only one file
- Zero installation needed
- Fast for small to medium applications
- Cross-platform (Windows, Linux, macOS, Android, iOS)
- Low memory + very small size
- Easy to learn
- Reliable — ACID compliant
- Open-source and free

**Disadvantages**

- Not ideal for very large applications
- No multi-user concurrency like MySQL or SQL Server
- Database file can get big
- Limited scalability
- Less secure for big corporate systems
- Not suitable for high-traffic websites
- No user management (only file-level permissions)



# Installing SQLite

Windows

Download SQLite Tools

Extract .exe files

Open CMD → run sqlite3

Linux/macOS

sudo apt install sqlite3
sqlite3

**Precompiled Binaries for Windows**

| | |
|---|---|
| sqlite-dll-win32-x86-3290000.zip (474.63 KiB) | 32-bit DLL (x86) for SQLite version 3.29.0. (sha1: 00435a36f5e6059287cde2cebb2882669cdba3a5) |
| sqlite-dll-win64-x64-3290000.zip (788.61 KiB) | 64-bit DLL (x64) for SQLite version 3.29.0. (sha1: c88204328d6ee3ff49ca0d58cbbee05243172c3a) |
| sqlite-tools-win32-x86-3290000.zip (1.71 MiB) | A bundle of command-line tools for managing SQLite database files, including the command-line shell program, the sqldiff.exe program, and the sqlite3_analyzer.exe program. (sha1: f009ff42b8c22886675005e3e57c94d62bca12b3) |

**Basic SQLite Commands**

Open/Create database

sqlite3 school.db

Show tables

.tables

Exit

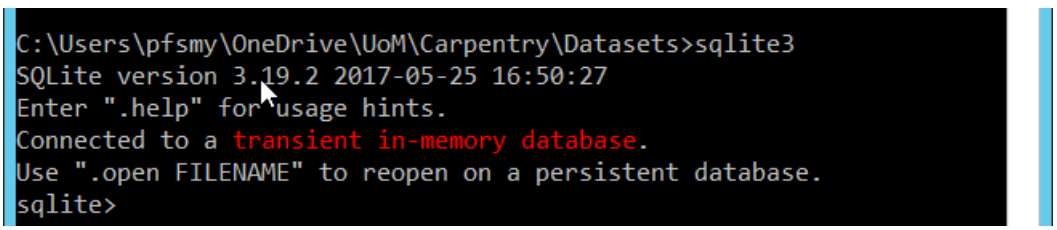
.quit

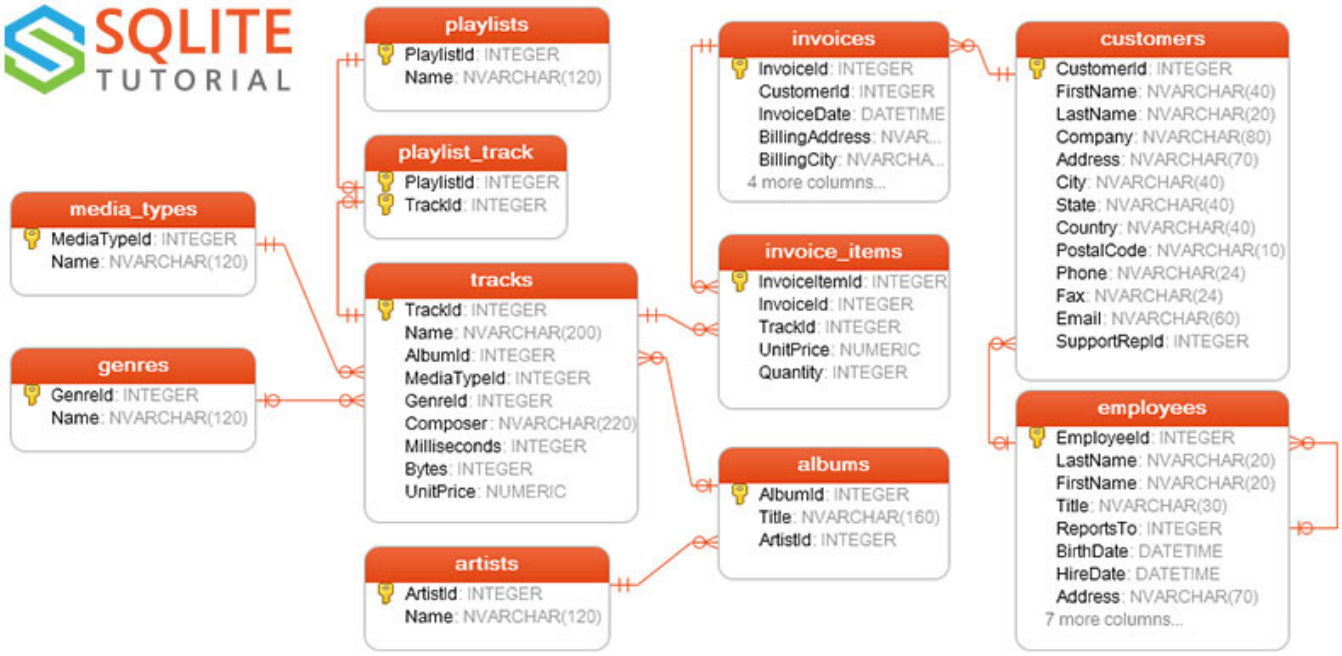
# SQLite Sample Database

Then, we will give you the links to download the sample database and its diagram. Finally, we'll show you how to connect to the sample database using the **sqlite3** tool.

We provide an SQLite sample database named **Chinook**, which is good for practicing with SQLite

The following database diagram shows the Chinook database tables and their relationships.






### Chinook sample database tables

The Chinook sample database contains 11 tables, as follows:

- employees table stores employee data such as id, last name, first name, etc. It also has a field named ReportsTo to specify who reports to whom.
- customers table stores customer data.
- invoices & invoice_items tables: these two tables store invoice data. The invoices table stores invoice header data and the invoice_items table stores the invoice line items data.
- artists table stores artist data. It is a simple table that contains the id and name.
- albums table stores data about a list of tracks. Each album belongs to one artist, but an artist may have multiple albums.
- media_types table stores media types such as MPEG audio and AAC audio files.
- playlists & playlist_track tables: playlists table stores data about playlists. Each playlist contains a list of tracks. Each track may belong to multiple playlists. The relationship between the playlists and tracks tables is many-to-many. The playlist_track table is used to reflect this relationship.

### How to connect to SQLite sample database

The sample database file is in ZIP format, so you'll need to extract it to a directory, such as **C:\sqlite\.** The file name is **chinook.db.**

First, open the Command Prompt on Windows or a Terminal on Unix-like systems and navigate to the SQLite directory where the sqlite3 (or **sqlite3.exe**) file is located.

```
sqlite3 chinook.db
```

It'll show something like this:

```
SQLite version 3.44.3 2024-03-24 21:15:01 (UTF-16 console I/O)
Enter ".help" for usage hints.
sqlite>
```

# SQLite Commands

Summary: in this tutorial, you will learn how to use the **sqlite3** command-line tool to interact with SQLite database files using commands.

The SQLite project offers a simple command-line tool called **sqlite3** (or **sqlite3.exe** on Windows), which allows you to interact with SQLite databases effectively.database files using commands.

### Opening sqlite3 tool

First, open your terminal.

Second, type the **sqlite3** command as follows:

```
sqlite3
```

Output:

```
SQLite version 3.44.3 2024-03-24 21:15:01 (UTF-16 console I/O)
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
```

By default, an SQLite session uses an in-memory database. It means that all the changes you make will be gone when the session ends.

The sqlite3 will start with the following command prompt waiting for your command:

```
sqlite>
```

### Getting helps

To show all available **sqlite3** commands and their purposes, you use the **.help** command as follows:

```
.help
```

Note that you don't add a semicolon (;) for the command that starts with the dot (.) like **.help**.

### Quitting sqlite3 tool

To exit the **sqlite3** program, you use the **.quit** command.

```
.quit
```

### Creating a new SQLite database

If you launch the **sqlite3** with an SQLite database file that does not exist, the **sqlite3** tool will create the database.

For example, the following command creates a database named sales in the **C:\sqlite\db** directory:

```
sqlite3 c:\sqlite\db\sales.db
```

Output:

```
SQLite version 3.44.3 2024-03-24 21:15:01 (UTF-16 console I/O)
Enter ".help" for usage hints.
sqlite>
```

**Opening a SQLite database file using sqlite3**

To open an existing database file, you use the **.open** command:

```
.open FILENAME
```

The **FILENAME** specifies the SQLite database you want to open.

For example, the following statement opens the **chinook.db** database located at **c:\sqlite\db\chinook.db**:

First, launch the **sqlite3** tool from your terminal:

```
sqlite3
```

Second, open the **chinook.db** database file:

```
.open c:\sqlite\db\chinook.db
```

If you want to open a specific database file when connecting to the SQLite database, you use the following command:

```
sqlite3 c:\sqlite\db\chinook.db
```

Output:

```
SQLite version 3.44.3 2024-03-24 21:15:01 (UTF-16 console I/O)
Enter ".help" for usage hints.
sqlite>
```

**Displaying the current database**

To show all databases in the current connection, you use the .databases command.

The **.databases** command displays at least one database with the name: **main** . For example, the following command shows all the databases of the current connection:

```
.databases
```

Output:

```
main: c:\sqlite\db\sales.db r/w
```

To add a database to the current connection, you use the statement **attach data** basecommand.

For example, the following command adds the **chinook** database to the current session.

```
attach database "c:\sqlite\db\chinook.db" AS chinook;
```

Now if you run the **.database** command again, the **sqlite3** returns two databases: **main** and **chinook**.

```
.databases
```

Output:

```
main: c:\sqlite\db\sales.db r/w
chinook: c:\sqlite\db\chinook.db r/w
```

To save time typing previous commands, you can use the up and down arrow keys to navigate between them.

**Showing tables in a database**

To display all the tables in the current database, you use the .tables command:

```
.tables
```

The following commands open a new database connection to the chinook database and display the tables:

Step 1. Open the **chinook** database:

```
sqlite3 c:\sqlite\db\chinook.db
```

Step 2. Show all the tables:

```
.tables
```

Output:

```
albums          employees       invoices        playlists
artists         genres          media_types     tracks
customers       invoice_items   playlist_track
```

If you want to find tables based on a specific pattern, you use the **.table** pattern command. The **sqlite3** uses the LIKE operator for pattern matching.

For example, the following statement returns the table that ends with the string **es**:

```
.table '%es'
```

Output:

```
employees       genres          invoices        media_types
```

**Showing table structures**

To display the structure of a table, you use the **.schema TABLE** command:

```
.schema TABLE
```

The **TABLE** argument can be the table name or a pattern that specifies one or more tables. For example, the following command shows the structure of the **albums** table:

```
.schema albums
```

Output:

```
CREATE TABLE IF NOT EXISTS "albums"
(
   [AlbumId] INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
   [Title] NVARCHAR(160)  NOT NULL,
   [ArtistId] INTEGER  NOT NULL,
   FOREIGN KEY ([ArtistId]) REFERENCES "artists" ([ArtistId])
            ON DELETE NO ACTION ON UPDATE NO ACTION
);
CREATE INDEX [IFK_AlbumArtistId] ON "albums" ([ArtistId]);
```

If you omit the **TABLE**, the **.schema** command will show the structures of all the tables.

```
.schema
```

To show the schema and the content of the **sqlite_stat** tables, you use the **.fullschema** command.

```
.fullschema
```

**Showing indexes**

To show all indexes of the current database, you use the **.indexes** command as follows:

```
.indexes
```

Output:

```
IFK_AlbumArtistId                IFK_PlaylistTrackTrackId
IFK_CustomerSupportRepId         IFK_TrackAlbumId
IFK_EmployeeReportsTo            IFK_TrackGenreId
IFK_InvoiceCustomerId            IFK_TrackMediaTypeId
IFK_InvoiceLineInvoiceId         sqlite_autoindex_playlist_track_1
IFK_InvoiceLineTrackId
```

If you are unfamiliar with indexes, you can learn about them in the **SQLite indexes** tutorial.

To show the indexes of a specific table, you use the **.indexes TABLE** command:

```
.indexes TABLE
```

For example, to show indexes of the **albums** table, you use the following command:

```
.indexes albums
```

Output:

```
IFK_AlbumArtistId
```

**Executing an SQL statement**

To execute an SQL statement, you type it in the **sqlite3>** command and hit the **Enter** key to start a new line:

```
sqlite> select *
   > (type the next clause)
```

As soon as you terminate the SQL statement with a semicolon ( **;** ), and hit the **Enter** key, **sqlite3** will execute the statement.

For example:

```
sqlite> select *
  ...> from albums
  ...> order by AlbumId
  ...> limit 3;
```

Output:

```
1|For Those About To Rock We Salute You|1
2|Balls to the Wall|2
3|Restless and Wild|2
```

The default output is not clear.

**Formatting query output**

To format the output of a query, you execute the following commands:

```
.mode column
.header on
.nullvalue NULL
```

- The **.mode** command sets the output mode to **column**.
- The **.header** command shows the column names in the output.
- The **.nullvalue** sets the literal string that represents **NULL**.

If you execute the above query again, you'll see the nicely formatted output:

```
sqlite> select *
   ...> from albums
   ...> order by AlbumId
   ...> limit 3;
```

Output:

```
AlbumId  Title
-------  -------------------------------------
1        For Those About To Rock We Salute You
2        Balls to the Wall
3        Restless and Wild
```

**Escaping the …> in the middle of the query**

Suppose you are typing a query and want to escape the …>, you can enter a semicolon ( ; ) and hit the Enter key.

For example:

```
sqlite> select *
   ...> from alien
   ...> ;
Parse error: no such table: alien
sqlite>
```

In this example, we found a mistake in the table name ( **alien** ) and wanted to terminate the statement earlier. We entered the semicolon ( **;** ) and hit the **Enter** key.

The **sqlite>** command prompt appears again waiting for the next command to execute.

**Saving the result of a query into a file**

To save the result of a query into a file, you use the **.output FILENAME** command.Enter key.

Once you issue the **.output** command, all the results of the subsequent queries will be saved to the file you specify in the **FILENAME** argument.

If you want to save the result of the next single query only to the file, you issue the **.once FILENAME** command.

To display the result of the query to the standard output again, you issue the **.output** command without arguments.

The following commands select the **title** from the **albums** table and write the result to the **albums.txt** file.

First, write the output of the query into the **c:/sqlite/db/output.txt** file:

```
.output c:/sqlite/db/output.txt
```

Second, execute a query that selects title from the **albums** table:

```
SELECT title FROM albums;
```

You can open the **output.txt** file to view the query result.

Finally, set the output back to the standard output:

```
.output
```

**Executing SQL statements from a file**

Suppose you have a file named **commands.txt** in the **c:\sqlite\db** folder with the following content:

```
SELECT albumid, title
FROM albums
ORDER BY title
LIMIT 10;
```

To execute the SQL statements in the **commands.txt** file, you use the **.read FILENAME** command as follows:

```
.read C:\sqlite\db\commands.txt
```

Output:

```
AlbumId     Title
----------  ----------------------
156         ...And Justice For All
257         20th Century Masters -
296         A Copland Celebration,
94          A Matter of Life and D
95          A Real Dead One
96          A Real Live One
285         A Soprano Inspired
139         A TempestadeTempestade
203         A-Sides
160         Ace Of Spades
```

If you see the following message, you need to double-check whether the file exists:

```
Error: cannot open "C:\sqlite\db\commands.txt"
```

In this tutorial, you have learned many useful commands in the **sqlite3** tool to perform various SQLite database tasks.

# Common Use Cases of SQLite (What it is usually used for)

SQLite is one of the most widely used embedded databases in the world. Its lightweight design, serverless architecture, and minimal configuration make it suitable for a wide range of practical applications. Below are the most common use cases of SQLite in real-world systems.

**1. Mobile Applications**

Mobile developers frequently choose SQLite because it is built directly into Android and iOS. It provides efficient data storage without the need for a separate server.

**Examples of usage:**

- Storing user profiles and app settings
- Saving messages, contacts, and call logs
- Offline caching of data for apps such as social networks, maps, and news apps

**2. Desktop Applications**

Many desktop software products use SQLite to handle local data because it is fast, stable, and does not require installation of a database server.

**Examples of desktop applications that use SQLite:**

- Google Chrome and Mozilla Firefox (history, cookies, bookmarks)
- Spotify (local music database)
- Adobe Lightroom (photo catalog storage)

SQLite works smoothly across Windows, macOS, and Linux.

**3. Embedded Systems and IoT Devices**

SQLite's small size makes it ideal for devices with limited memory and computing power, such as embedded systems and Internet of Things (IoT) devices.

**Typical devices using SQLite:**

- Smart TVs
- Smart watches
- Routers
- Home appliances
- Automotive infotainment systems

Because these devices do not run full database servers, SQLite is the perfect match.

**4. Standalone Applications and Local Tools**

Some applications are built to function independently without internet access or server dependency. SQLite is often used in:

- Personal finance apps
- Note-taking apps
- POS (Point of Sale) systems
- Inventory management tools
- Library catalog systems

Developers choose SQLite because the database is simply a single file that can easily be copied, backed up, or moved.

**5. Web Development Prototyping**

SQLite is commonly used in the early stages of web application development. Developers use it to quickly create and test database structures before switching to larger DBMS such as MySQL, PostgreSQL, or SQL Server.

**Reasons for using SQLite during development:**

- Easy setup
- Zero configuration
- Ideal for quick experiments and sample projects
- Perfect when teaching database concepts

Frameworks like Django, Laravel, and Ruby on Rails often default to SQLite for development mode.

**6. Data Analysis and Local Data Storage**

SQLite is also used by data analysts and researchers who need a lightweight database for storing datasets locally.

**Uses include:**

- Storing medium-sized datasets
- Running SQL queries without a server
- Managing CSV or JSON data
- Local data cleaning and transformation

SQLite is often used with Python, especially in data science workflows using pandas and Jupyter notebooks.

### 7. File-Based Data Formats

Because an SQLite database is just a single file, many applications use it as a structured alternative to XML or JSON for internal data storage.

**Advantages of file-based databases with SQLite:**

- Portable
- Easy to backup
- Easy to share
- Safe and ACID-compliant

Many apps store their entire data configuration in a single .db or .sqlite file.

### 8. Applications Needing Offline Capability

Apps that must work without an internet connection depend on SQLite for storing all local data.

**Example offline apps:**

- Offline dictionaries
- Translation apps
- Offline map/navigation apps
- Learning apps (e-learning, quiz systems)
- Attendance systems

SQLite ensures smooth operation even without a network connection.

### 9. Small Websites and Backend Systems

While SQLite is not ideal for high-traffic websites, it is perfect for smaller projects such as:

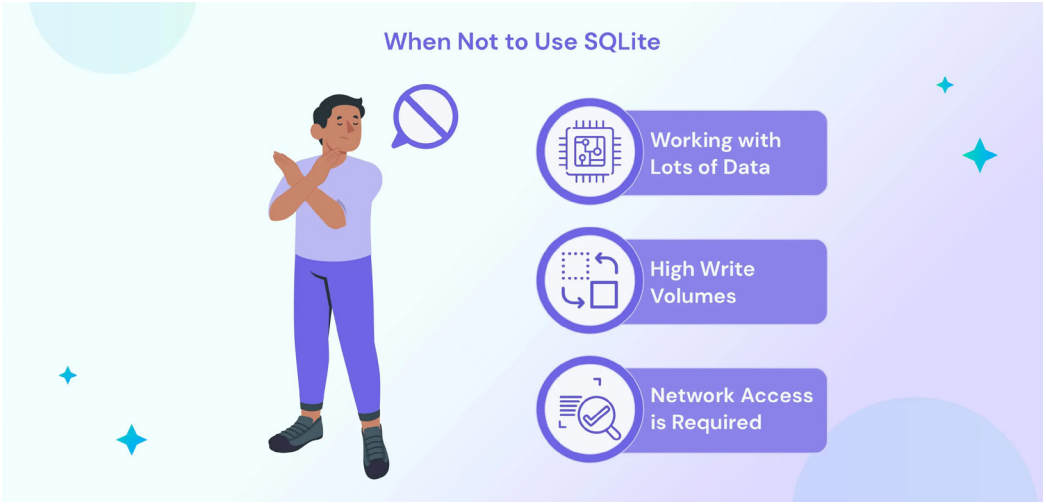
- Personal websites
- Portfolio websites
- Low-traffic APIs
- Local CMS systems
- Internal company tools

Because it does not require a server, developers can deploy small web platforms very quickly.

SQLite is used everywhere—from mobile phones and desktop software to IoT devices and scientific research tools. Its simplicity, portability, and serverless design make it one of the most versatile database engines available today.



# When not to use SQLite




When working with data, SQLite is a handy tool. It is lightweight, versatile, and easy to use, making it the ideal choice for many applications. However, there are certain situations where there are better choices than this, and you should consider using another database system. In this section, we will discuss three conditions when you should avoid using SQLite: working with lots of data, high write volumes, and network access is required.

### 1. Working with lots of data

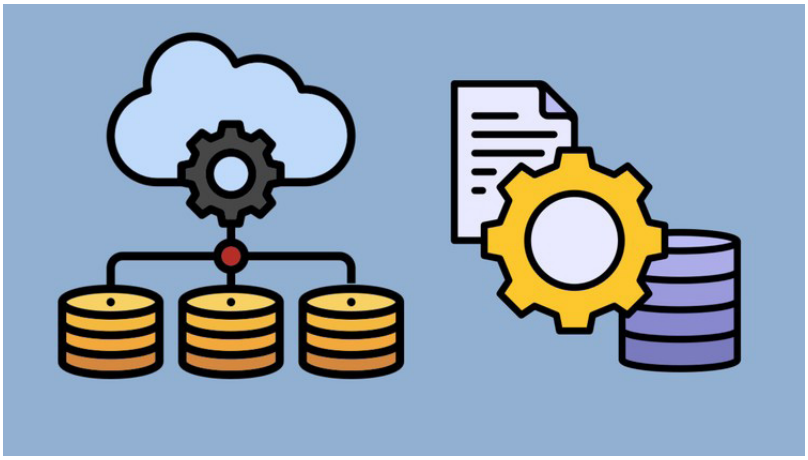
When dealing with large amounts of data, SQLite may struggle to keep up. SQLite is best suited for applications that store and access relatively small amounts of data (up to several gigabytes). If you are dealing with larger amounts of data, consider using one of the more powerful database systems, such as MySQL or PostgreSQL. These systems are designed to handle larger amounts of data and can even scale to handle large workloads.

### 2. High write volumes

The second situation when you should avoid using SQLite is when dealing with high write volumes. SQLite is not optimised for large write volumes and can quickly become overwhelmed. Consider using a database system explicitly designed for high-write workloads in these cases, such as MongoDB or Redis. These systems are designed to handle large write volumes and can scale to handle more load as needed.

### 3. Network access is required

SQLite is not the ideal choice if your application requires network access. SQLite works best when it is used locally, as it cannot be accessed remotely. Consider using a database system designed for distributed access, such as Cassandra or MongoDB. These systems are designed to handle distributed queries and can be accessed from multiple locations.



# What is SQLite used for

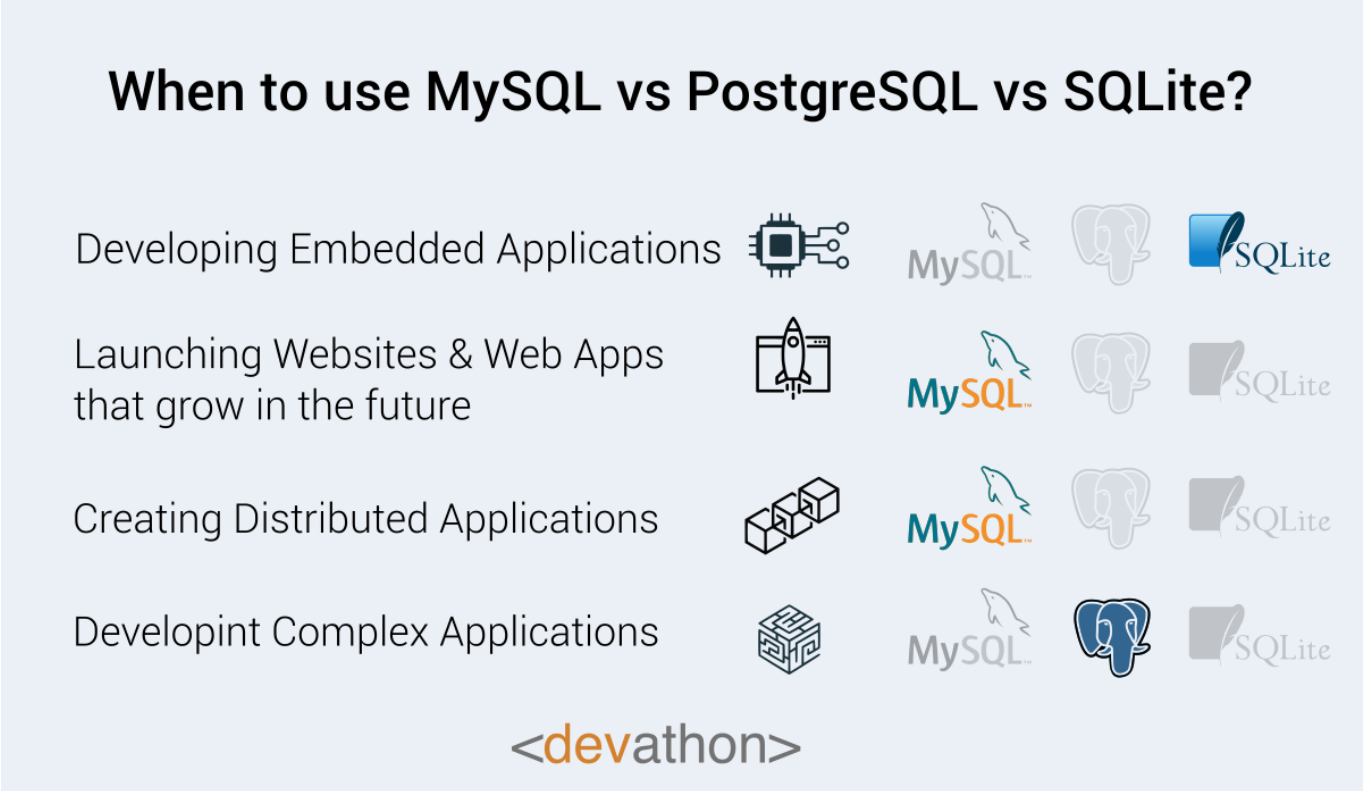



You would use SQLite to create applications like version control systems or similar desktop applications. Media editing tools, financial analysis tools, and record-management programs are a few more examples where you would use SQLite.

The following are examples of companies that use SQLite:

1. Hyatt Hotels Corporation.
2. SAS.
3. Airgas.
4. Internet Brands, Inc.

# When to use MySQL vs PostgreSQL vs SQLite



Now that you know enough about MySQL vs PostgreSQL vs SQLite, let's see the scenarios where you should one or the other RDBMS:

Developing embedded applications: Are you creating embedded applications requiring portability. Single-user apps are examples of these. These apps won't need expansion in the future. Use SQLite, which is best for embedded apps. This lightweight RDBMS delivers the optimal performance for such apps.

Creating distributed applications: Choose MySQL if you are creating distributed apps. The replication support offered by MySQL makes it the ideal choice for this.




Before open-source RDBMSs like MySQL, PostgreSQL, and SQLite became popular, organizations used traditional SQL databases. We now briefly talk about the following popular traditional RDBMSs.

## Conclusion & Summary

We provided an overview of MySQL, PostgreSQL, and SQLite. We talked about their pros and cons, and we compared them. Subsequently, we talked about self-hosted vs cloud-hosted RDBMSs. Finally, we briefly discussed a few popular licensed RDBMSs and cloud-based RDBMSs. Analyze your project requirements carefully before you choose an RDBMS.

Are you looking to get your App built? Contact us at hello@devathon.com or visit our website Devathon to find out how we can breathe life into your vision with beautiful designs, quality development, and continuous testing.


DATA-01
DATA-03
DATA-01
DATA-03